

SENTINEL-X

L'avant-poste industriel du futur

Rapport d'ingénierie technique

Schéma réseau · câblage électronique · matrice de sécurité
documentation IA · gabarit d'audit de sécurité · MCO · poster A3

LIVRABLE	Workshop2026-M1-GX-Dossier.pdf
CONSORTIUM	Groupe GX — numéro à confirmer (kit cyber : g11)
VARIANTE	Option B — topologie Edge-to-Server, PC portable d'un apprenant
MEMBRES	[À COMPLÉTER : prénom NOM — filière, ×4 à 6]
VERSION	v1.0 — jeudi 8 octobre 2026 (gel du code)
DÉPÔT	github.com/turbomulot/... — branches nino, backend, c_plus_plus, cyber, IA-*

« Sentinel-X : la sécurité à la bordure. »

Les encadrés orange [À COMPLÉTER] signalent les informations à renseigner par l'équipe (résultats d'audit, mesures finales, noms).

Sommaire

1. Synthèse	4
Conventions du document	4
2. Contexte et exigences	5
2.1 Traçabilité des exigences	5
2.2 Choix de l'option B	5
3. Architecture globale et réseau	6
3.1 Vue d'ensemble	6
3.2 Schéma réseau et plan d'adressage	7
3.3 Composants logiciels et ports	8
3.4 Matrice des flux	9
4. Boîtier et électronique	10
4.1 Schéma de câblage	10
4.2 Affectation des broches	10
4.3 Alimentation et budget de courant	11
4.4 Points de vigilance	11
4.5 Fabrication (Fablab)	11
5. Firmware ESP8266	12
5.1 Cycle de fonctionnement	12
5.2 Messages publiés	12
5.3 Version sécurisée (cible MQTTS)	12
5.4 Revue du code actuel	12
6. Architecture logicielle et flux de données	14
6.1 Topics MQTT	14
6.2 API REST (préfixe /api/v1)	14
6.3 Modèle de données	15
6.4 Dashboard	15
7. Matrice de sécurité	16
7.1 Chiffrement et gestion des certificats	16
7.2 Durcissement du serveur	16
7.3 Contrôle d'accès applicatif	17
7.4 Analyse STRIDE	17
7.5 Failles relevées dans le code (audits du 6 et du 8 octobre)	17

7.6 Preuve matérielle du chiffrement	17
8. IA vision : détection de présence	19
8.1 Modèle et réglages	19
8.2 Performances	19
8.3 Intégration et vie privée	19
9. IA prédictive : maintenance par détection d'anomalies	21
9.1 Pourquoi pas de seuils statiques	21
9.2 Variables (features)	21
9.3 Entraînement sur les données réelles	21
9.4 Évaluation par injection de dérives lentes	22
9.5 Limites et suite	23
10. Gabarit de rapport d'audit de sécurité	24
10.1 Identification de l'audit	24
10.2 Grille des points de contrôle	24
10.3 Synthèse des constats	25
10.4 Échelle de gravité	25
10.5 Indicateurs avant / après durcissement	25
10.6 Recommandations défensives génériques	25
11. Supervision et maintien en condition opérationnelle	27
11.1 Indicateurs suivis	27
11.2 Résilience déjà en place	27
11.3 Gestion des journaux et des données	27
11.4 Procédures d'exploitation	28
12. Conclusion et perspectives	29
Perspectives	29
Annexe — Glossaire	29
Annexe — Poster A3	30

1. Synthèse

SENTINEL-X est un prototype cyber-physique de surveillance d'avant-poste pour les micro-centrales d'AetherCorp. Un boîtier autonome à base d'ESP8266 mesure température, humidité, gaz combustibles et présence, affiche son état sur un écran OLED et pilote un buzzer et deux LED. Il transmet ses mesures en Wi-Fi au **PC Serveur Local**, ici le **PC portable d'un apprenant (option B)**, qui héberge la pile conteneurisée (Mosquitto, API Node.js, PostgreSQL, proxy HTTPS), la vision par ordinateur sur la webcam USB et la maintenance prédictive.

Ce dossier décrit le système tel qu'il existe dans le dépôt Git du groupe au 8 octobre 2026 et la configuration cible durcie préparée par l'équipe CYBER. Chaque fois que l'état réel diffère de la cible, l'écart est indiqué explicitement.

Brique	Technologie (dépôt)	État au 08/10	Où
Firmware boîtier	C++ Arduino, PlatformIO (nodemcu2), PubSubClient, ArduinoJson 6	Fonctionnel en MQTT clair ; version MQTTS fournie par la cyber	branche c_plus_plus
Backend / API	Node.js 20, Express 4, socket.io, MQTT.js, pg	Fonctionnel (REST + temps réel)	backend/
Base de données	PostgreSQL 16 (Docker) + SQLite (vision)	Fonctionnelle, dumps du 08/10	backend/dumps/
Dashboard	React 18 + Vite 5, Recharts, socket.io-client	Fonctionnel (supervision, commandes, badges)	frontend/
Vision IA	Python 3.13, OpenCV, Ultralytics YOLOv8n, Flask	Fonctionnelle ; POST /alerts à brancher	webcam-detection/
IA prédictive	pandas, scikit-learn IsolationForest	Entraînée sur données réelles	branche IA-maintenance-predictive
Sécurité	OpenSSL (PKI ECDSA), Mosquitto TLS + ACL, UFW, sshd, Docker	Kit fourni et testé ; à déployer sur le serveur	branche cyber, security/

Chiffres clés

5 429 mesures réelles et **433** alertes enregistrées entre le 07/10/2026 14:10 et le 08/10/2026 12:11 · **8** flux réseau documentés, **2** ports exposés sur le Wi-Fi de table · Isolation Forest à **10** variables cinétiques : **6/6** dérives lentes détectées contre **1/6** pour un seuil statique.

À COMPLÉTER PAR L'ÉQUIPE

Numéro de groupe définitif (remplacer GX dans les noms de fichiers et g11 dans les topics/comptes si différent), noms et filières des membres, état réel de la migration MQTTS au moment du dépôt.

Conventions du document

- Le numéro de groupe n'étant pas confirmé, les livrables portent **GX**. Le plan d'adressage reprend la valeur **11** du kit CYBER (security/.env.example : GROUP_NUMBER=11, SERVER_IP=10.10.11.1).
- Les chemins de fichiers sont relatifs à la racine du dépôt. Les branches citées sont celles du dépôt distant.
- Prouvé** : vérifié et preuve conservée · **Fourni** : livré dans le dépôt, à vérifier une fois déployé · **Exigence** : règle à respecter par une autre filière.

2. Contexte et exigences

En 2050, AetherCorp Industrial Solutions déploie des micro-centrales énergétiques dans des zones isolées et hostiles. Elles subissent trois menaces simultanées : cyberattaques de déstabilisation réseau, intrusions physiques d'espionnage industriel et risques environnementaux (fuites de gaz, surchauffes). Faute de personnel sur place, SENTINEL-X doit détecter, alerter et résister seul, avec un Centre de Commandement Tactique matérialisé par un PC Serveur Local durci.

2.1 Traçabilité des exigences

Exigence du sujet	Réponse SENTINEL-X	\$
ESP8266 unique, firmware C++ (Arduino / PlatformIO)	NodeMCU v2, projet PlatformIO c++/arduino, lecture cadencée 2 s	5
DHT22, MQ-2, PIR HC-SR501, OLED I2C, buzzer, LED	Câblage complet, pont diviseur sur A0, transistor buzzer	4
Chiffrement de bout en bout (MQTTS / HTTPS)	Mosquitto 8883 TLS 1.2, CA ECDSA du groupe ; Caddy 443	7
API REST / WebSocket, POST /api/v1/alerts	Express + socket.io, 5 routes /api/v1, validation stricte	6
Dashboard temps réel + contrôle des actionneurs	React : courbes, statut, flux webcam, buzzer et LED	6
Vision : webcam USB sur le serveur, < 100 ms / trame, 640×480	YOLOv8n, redimensionnement 640×480, fil de capture dédié	8
Prédictif sans « if temp > 40 »	Isolation Forest sur 10 variables cinétiques	9
Docker-Compose : BDD + API + Mosquitto	4 services, réseau interne, secrets Docker	3, 6
Point d'accès de table, plan IP, isolation	10.10.11.0/24, hostapd/dnsmasq, ap_isolate, pas de routage	3
UFW, SSH par clés, Docker non-root	Scripts security/hardening/, contrôles self-audit.sh	7
Audit de sécurité (journée du jeudi)	Gabarit d'audit défensif à compléter, recommandations de durcissement	10
MCO : CPU/RAM, volumes de logs MQTT	Indicateurs, seuils, rotation des logs, runbook	11

2.2 Choix de l'option B

Le groupe a retenu l'**option B** : le Raspberry Pi est retiré et le rôle de PC Serveur Local est tenu par le portable d'un apprenant. Le boîtier ne contient que l'ESP8266 et ses composants.

Critère	Option A — Raspberry Pi 5	Option B — PC portable (retenue)
Puissance IA	CPU ARM 4 cœurs, YOLO lent sans export NCNN	Intel Core Ultra 7 155U, 16 Go : YOLOv8n en PyTorch CPU
Webcam USB	Sur le Pi, dans le boîtier	Directement sur le portable, orientable
Point d'accès Wi-Fi	hostapd sur le Pi	Carte Wi-Fi du portable (hostapd ou point d'accès mobile)
Intégration	Boîtier plus gros, alimentation 5 V / 5 A	Boîtier compact : ESP8266 + capteurs
Risque	Carte et alimentation à fiabiliser	Le portable est un poste personnel à durcir et isoler

3. Architecture globale et réseau

3.1 Vue d'ensemble

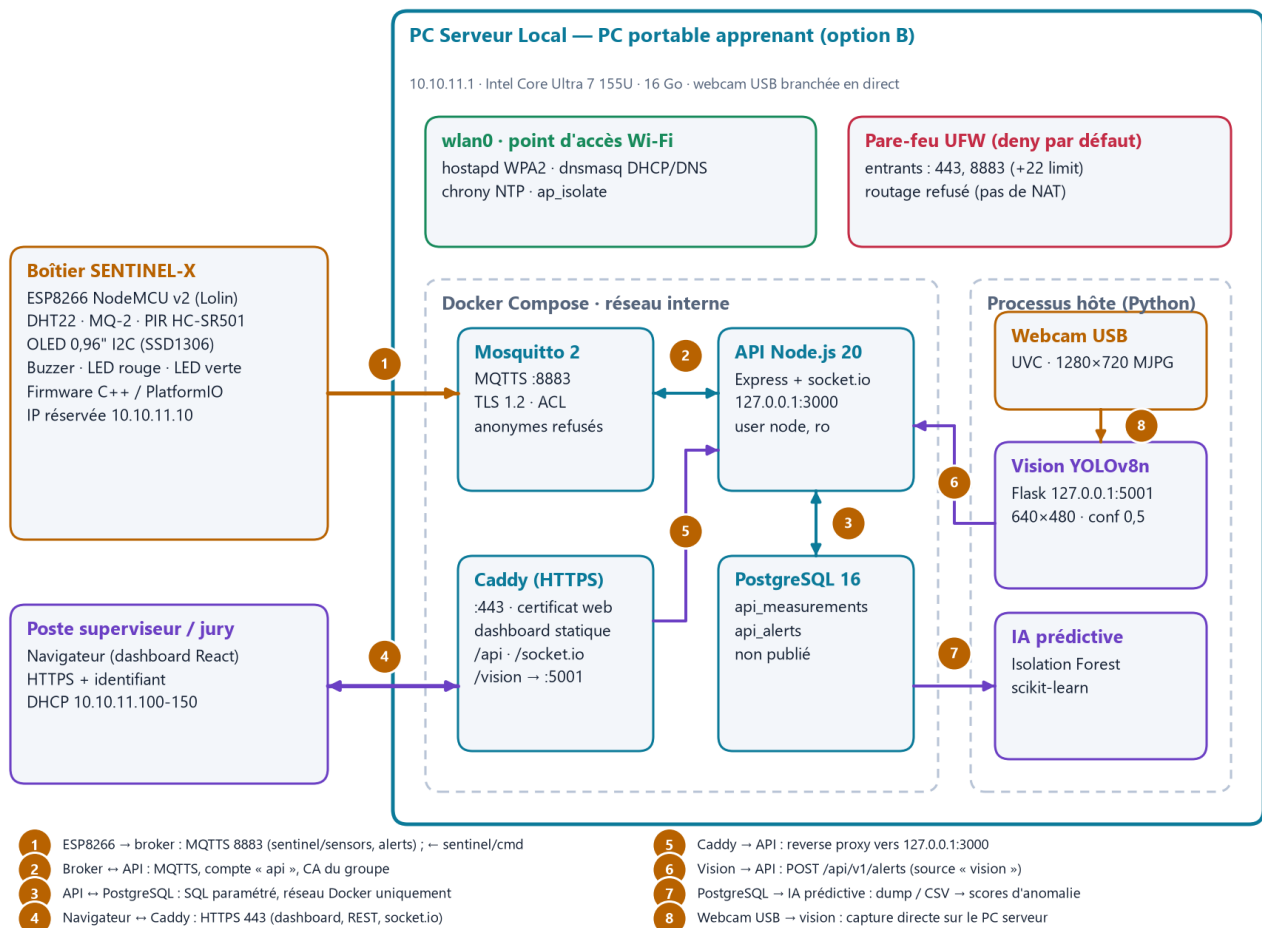


Figure 1 — Architecture Edge-to-Server (option B). Les numéros renvoient à la matrice des flux du § 3.4.

Les services réseau tournent dans Docker Compose sur un réseau interne. Seuls Caddy (443) et Mosquitto (8883) sont publiés, et uniquement sur l'adresse du Wi-Fi de table. La vision tourne en processus hôte car elle doit ouvrir la webcam USB : elle écoute sur 127.0.0.1:5001 et n'est accessible du réseau qu'à travers le proxy HTTPS.

3.2 Schéma réseau et plan d'adressage

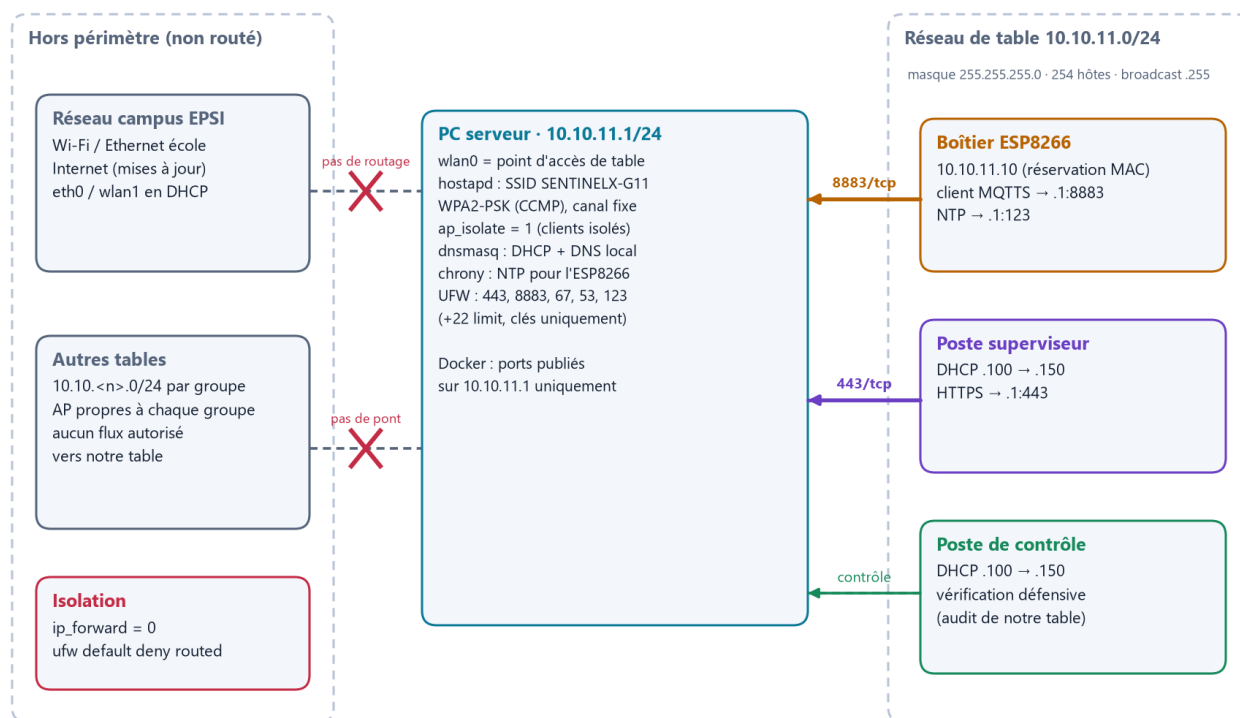


Figure 2 — Réseau de table étanche 10.10.11.0/24 et isolation du campus.

Élément	Adresse / plage	Masque	Attribution	Rôle
Réseau de table	10.10.11.0/24	255.255.255.0	—	Sous-réseau dédié au groupe (11 = n° kit)
PC serveur (wlan0)	10.10.11.1	/24	statique	Point d'accès, passerelle, DNS, NTP, broker, HTTPS
Boîtier ESP8266	10.10.11.10	/24	réservation DHCP (MAC)	Client MQTTS, NTP
Postes superviseur / jury	10.10.11.100 → .150	/24	DHCP dnsmasq, bail 12 h	Navigateur HTTPS
Réserve équipements	10.10.11.2 → .99	/24	statique	Futurs boîtiers, poste d'audit
Broadcast	10.10.11.255	—	—	—
Interface campus	DHCP école	—	eth0 / 2e carte	Mises à jour uniquement, jamais routée
Réseau Docker	bridge interne	—	Docker	db non publiée, API sur 127.0.0.1

Point d'accès de table (proposition de configuration)

```
# /etc/hostapd/hostapd.conf
interface=wlan0
ssid=SENTINELX-G11
hw_mode=g
channel=6                # canal fixe, à coordonner entre tables
wpa=2
wpa_key_mgmt=WPA-PSK
rsn_pairwise=CCMP
wpa_passphrase=<secret, hors Git>
ap_isolate=1             # les clients ne se voient pas entre eux

# /etc/dnsmasq.d/sentinel.conf
interface=wlan0
dhcp-range=10.10.11.100,10.10.11.150,255.255.255.0,12h
dhcp-host=<MAC de l'ESP8266>,10.10.11.10
dhcp-option=option:ntp-server,10.10.11.1
```

Isolation des flux

- **Aucun routage** : `net.ipv4.ip_forward=0` et `ufw default deny routed`. Le serveur ne fait ni NAT ni pont vers le campus : un client de la table n'atteint jamais une autre table.
- **ap_isolate=1** : un poste connecté au Wi-Fi de table ne peut pas parler directement à l'ESP8266 (bloque l'ARP spoofing client-à-client).
- **Ports Docker publiés sur 10.10.11.1 uniquement** (ex. `10.10.11.1:8883:8883`) : un port publié par Docker contourne UFW, d'où cette règle.
- **Canal Wi-Fi fixe** et SSID propre au groupe pour éviter les interférences entre tables.

Écart prototype / cible

Le firmware actuel vise `192.168.137.1:1883`, l'adresse par défaut du **point d'accès mobile Windows** du portable (réseau `192.168.137.0/24`). C'est la configuration de développement de mardi-mercredi. La cible du présent dossier est `10.10.11.0/24` avec MQTTs. Le passage impose de régénérer les certificats si l'IP change (elle figure dans le SAN).

À COMPLÉTER PAR L'ÉQUIPE

Préciser l'OS réellement utilisé par le PC serveur le jour de la démo (Windows 11 + Docker Desktop ou Linux Ubuntu/Arch) et la solution de point d'accès retenue (hostapd ou point d'accès mobile). Les scripts UFW/sshd du dépôt visent Linux ; sous Windows, transposer avec le pare-feu Windows Defender (règles entrantes `443/8883` sur le profil du point d'accès uniquement).

3.3 Composants logiciels et ports

Composant	Image / exécution	Écoute	Exposition	Compte
Mosquitto	eclipse-mosquitto:2	8883/tcp (TLS)	10.10.11.1	uid 1883
API	node:20-alpine (backend/Dockerfile)	3000/tcp	127.0.0.1 → Caddy	node (non-root), read_only
PostgreSQL	postgres:16-alpine	5432/tcp	réseau Docker seulement	postgres (abandon des droits)
Caddy	proxy HTTPS (pile INFRA)	443/tcp	10.10.11.1	non-root, cap NET_BIND_SERVICE
Vision	Python 3.13 hôte (stream_detection.py)	5001/tcp	127.0.0.1 → Caddy	utilisateur courant
Dashboard	build Vite statique servi par Caddy	via 443	10.10.11.1	—

3.4 Matrice des flux

#	Source → destination	Port / proto	Contenu	Protection
1	ESP8266 → Mosquitto	8883/tcp MQTTS	sentinel/sensors, sentinel/alerts ; retour sentinel/cmd	TLS 1.2, CA du groupe, compte esp-g11, ACL
2	Mosquitto ↔ API	8883 (Docker)	abonnement sentinel/#, publication cmd	TLS, compte api, MQTT_CA_FILE
3	API ↔ PostgreSQL	5432 (Docker)	INSERT / SELECT mesures et alertes	réseau interne, requêtes paramétrées, secret Docker
4	Navigateur ↔ Caddy	443/tcp HTTPS	dashboard, /api/*, /socket.io/*, /vision/*	certificat web, identifiant
5	Caddy → API	127.0.0.1:3000	reverse proxy REST + WebSocket	boucle locale uniquement
6	Vision → API	127.0.0.1:3000	POST /api/v1/alerts (source vision)	boucle locale
7	PostgreSQL → IA prédictive	fichier	pg_dump → CSV → modèle	fichier local, hors Git
8	Webcam → vision	USB (UVC)	1280×720 MJPG	physique, pas de réseau
—	ESP8266 → serveur	123/udp, 67/udp, 53	NTP (indispensable au TLS), DHCP, DNS	Wi-Fi de table uniquement
—	Admin → serveur	22/tcp (option)	SSH	clé ed25519, ufw limit, fail2ban

Fermés depuis le réseau : 1883 (MQTT clair), 3000 (API), 5001 (vision), 5432 (base), 5173 (serveur de développement Vite). La vérification de cette surface d'exposition réduite (seuls 443 et 8883 répondent) est le premier contrôle du gabarit d'audit défensif (§ 10).

4. Boîtier et électronique

4.1 Schéma de câblage

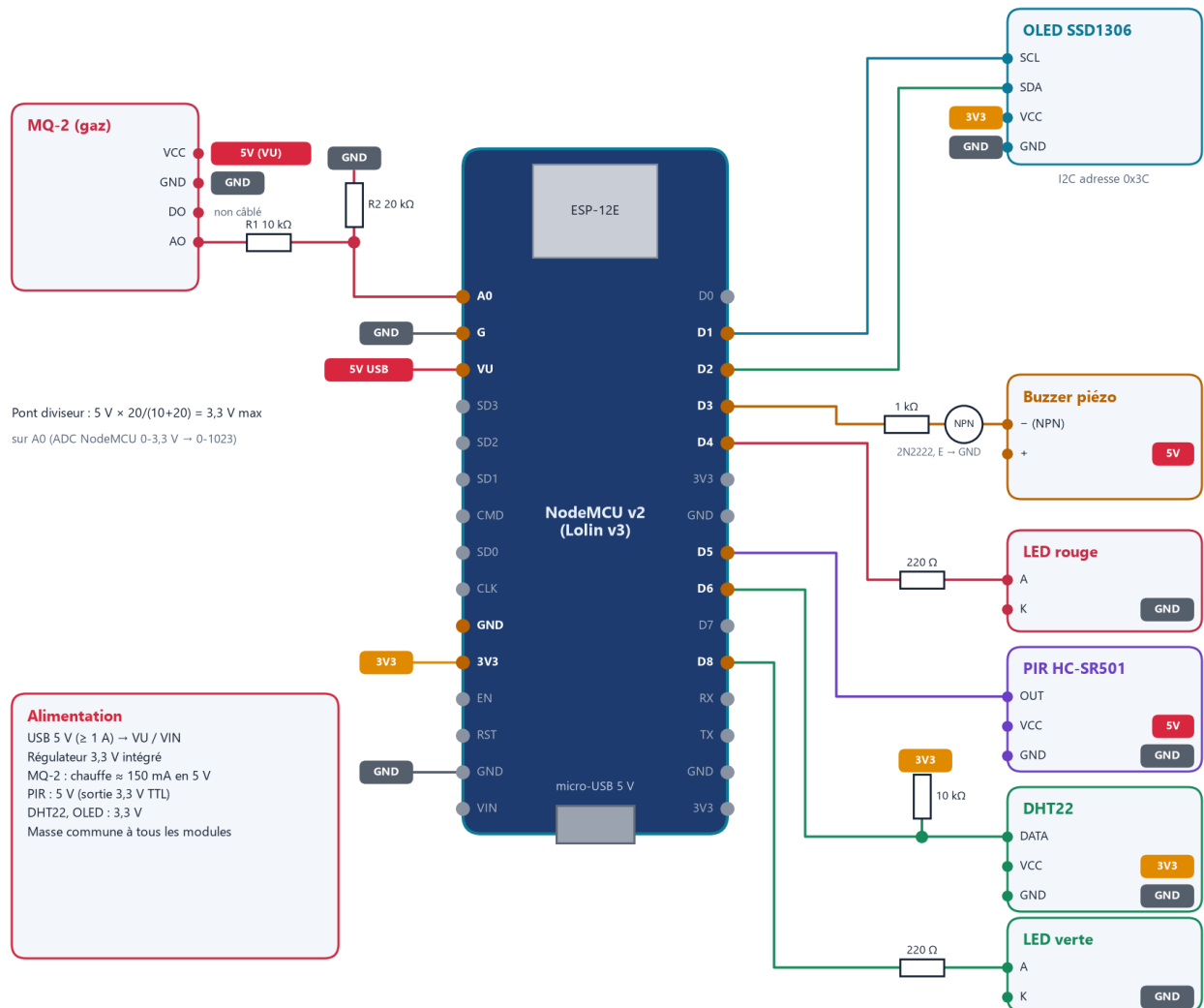


Figure 3 — Câblage NodeMCU v2 (broches du firmware, branche c_plus_plus).

4.2 Affectation des broches

Composant	Broche module	NodeMCU	GPIO	Tension	Remarque
DHT22	DATA	D6	GPIO12	3,3 V	Pull-up 10 kΩ vers 3V3 (souvent présent sur module)
MQ-2	AO	A0	ADC0	5 V (chauffe)	Pont 10 kΩ / 20 kΩ : 0-5 V → 0-3,3 V ; DO non utilisé
PIR HC-SR501	OUT	D5	GPIO14	5 V	Sortie 3,3 V TTL ; temporisation et sensibilité réglées par potentiomètres
OLED SSD1306	SDA / SCL	D2 / D1	GPIO4 / GPIO5	3,3 V	I2C par défaut de Wire.begin(), adresse 0x3C, 128×64
Buzzer	– via NPN	D3	GPIO0	5 V	1 kΩ sur la base d'un 2N2222, émetteur à la masse
LED rouge	anode	D4	GPIO2	3,3 V	220 Ω en série, cathode à la masse
LED verte	anode	D8	GPIO15	3,3 V	220 Ω en série, cathode à la masse

4.3 Alimentation et budget de courant

Consommateur	Typique	Pointe	Source
ESP8266 (Wi-Fi actif, TLS)	80 mA	170 mA (émission)	régulateur 3,3 V de la carte
MQ-2 (élément chauffant)	150 mA	180 mA	VU / VIN 5 V
OLED 0,96"	10 mA	20 mA	3V3
DHT22 + PIR	< 2 mA	2,5 mA	3V3 / 5 V
LED x2 (220 Ω)	2 x 6 mA	12 mA	GPIO
Buzzer piézo actif	25 mA	30 mA	5 V via NPN
Total	≈ 280 mA	≈ 415 mA	USB 5 V ≥ 1 A suffisant

4.4 Points de vigilance

- **Broches de démarrage** : D3 (GPIO0) et D4 (GPIO2) doivent être à l'état haut au boot, D8 (GPIO15) à l'état bas. La LED verte sur D8 tire bien vers la masse. En revanche la base du transistor sur D3 et la LED sur D4 peuvent tirer ces broches vers le bas et empêcher le démarrage (ou forcer le mode flash). Si le boîtier ne redémarre pas seul, déplacer le buzzer sur D0 (GPIO16) et la LED rouge sur D7 (GPIO13), deux broches libres, puis mettre à jour les constantes du firmware.
- **Entrée analogique** : sans pont diviseur, la sortie AO du MQ-2 alimenté en 5 V dépasse la plage 0-3,3 V de la NodeMCU. Le pont 10 k Ω / 20 k Ω ramène 5 V à 3,33 V.
- **MQ-2** : préchauffage d'1 à 2 minutes avant des valeurs stables (24 h pour un étalonnage). Les valeurs transmises sont des lectures ADC brutes 0-1023, pas des ppm : le libellé « ppm » du dashboard (frontend/src/config.js) est à corriger ou à justifier par un étalonnage.
- **DHT22** : une lecture toutes les 2 s minimum, ce que respecte la cadence du firmware.
- **PIR** : environ 60 s de stabilisation à la mise sous tension ; régler la temporisation au minimum pour une détection réactive.

4.5 Fabrication (Fablab)

Coque conçue sous Fusion 360, imprimée sur Creality K2 Plus : logement de la NodeMCU et de la breadboard, fenêtre pour l'OLED, ouïes d'aération devant le MQ-2 et le DHT22 (éloignés de la chauffe du MQ-2), dôme du PIR dégagé, passe-câble USB unique. Façade gravée au laser (Creality Falcon A1) : logo AetherCorp, consignes de sécurité et numéro de série.

À COMPLÉTER PAR L'ÉQUIPE

Photos du boîtier fini, dimensions, matériau (PLA / PETG), numéro de série gravé, temps d'impression.

5. Firmware ESP8266

Le firmware (c++/arduino/src/main.cpp, branche c_plus_plus) est un projet PlatformIO pour la carte nodemcu2. Dépendances : Adafruit SSD1306 2.5, Adafruit GFX 1.11, DHT sensor library 1.4, PubSubClient 2.8, ArduinoJson 6.21.

5.1 Cycle de fonctionnement

- **setup()** : initialisation I2C et DHT, broches en sortie à l'état bas, écran de démarrage en trois étapes (Wi-Fi, synchronisation NTP, serveur MQTT) affiché sur l'OLED.
- **loop()** : maintien de la connexion MQTT (reconnexion avec identifiant aléatoire SentinelNode-xxxx), puis toutes les 2 s : lecture des quatre capteurs, publication de la mesure, publication d'alertes locales, rafraîchissement de l'OLED (Gaz, Hum, Tmp, Mvt).
- **receptionCommande()** : abonné à sentinel/cmd, applique {"buzzer": bool, "led": "red" | "green" | "off"}.

5.2 Messages publiés

```
sentinel/sensors {"temp": 24.5, "hum": 52.1, "gas": 120, "pir": 0, "ts": 1791280000}
sentinel/alerts {"type": "gas", "level": "warn" | "critical", "value": 640}
sentinel/alerts {"type": "motion", "level": "warn", "value": 1}
sentinel/cmd <- {"buzzer": true, "led": "red"}
```

Les alertes locales du firmware (gaz > 400 avertissement, > 700 critique ; mouvement) sont des **alarmes de repli** : elles déclenchent l'affichage et le signal physique même si le serveur est injoignable. La détection d'anomalies, elle, est confiée au modèle prédictif (§ 9), conformément à l'interdiction des seuils statiques.

5.3 Version sécurisée (cible MQTTS)

L'équipe CYBER fournit les règles et le code de connexion (security/docs/tls-esp8266.md) :

Règle	Mise en œuvre
Confiance limitée à la CA du groupe	BearSSL::X509List + setTrustAnchors(), jamais setInsecure()
Heure valide avant le TLS	configTime(0, 0, SERVER_IP), redémarrage si pas d'heure en 30 s
Secrets hors Git	include/secrets.h (ignoré), seul secrets.example.h est versionné
Authentification	compte esp-g11, mot de passe aléatoire 128 bits
Présence fiable	Last Will : le broker publie « offline » sur status si le boîtier disparaît
Anti-rejeu	champ seq + ts dans chaque mesure
Commandes	liste blanche (BUZZER_ON, LED_RED_ON...), tout le reste ignoré
RAM	probeMaxFragmentLength() puis tampons TLS 1024 octets (≈ 25 Ko économisés)

5.4 Revue du code actuel

#	Constat (branche c_plus_plus, commit 5738b71)	Impact	Correction
F1	SSID et clé Wi-Fi écrits en clair dans main.cpp, donc dans l'historique Git	Élevé	Changer la clé du point d'accès, passer par secrets.h
F2	MQTT en clair sur 1883, connexion anonyme	Élevé	Version MQTTS du kit cyber (§ 5.3)
F3	Ligne isolée «) » après la boucle de connexion Wi-Fi	Compilation	À vérifier : supprimer la ligne
F4	Alerte motion republiée toutes les 2 s tant que le PIR est haut	Bruit (433 alertes)	Publier sur front montant uniquement
F5	NTP sur pool.ntp.org, injoignable sur réseau isolé	TLS impossible	NTP = 10.10.11.1 (chrony)
F6	Valeur 0 envoyée si le DHT22 échoue (isnan → 0)	Fausse mesures	Envoyer null ; le backend l'accepte

6. Architecture logicielle et flux de données

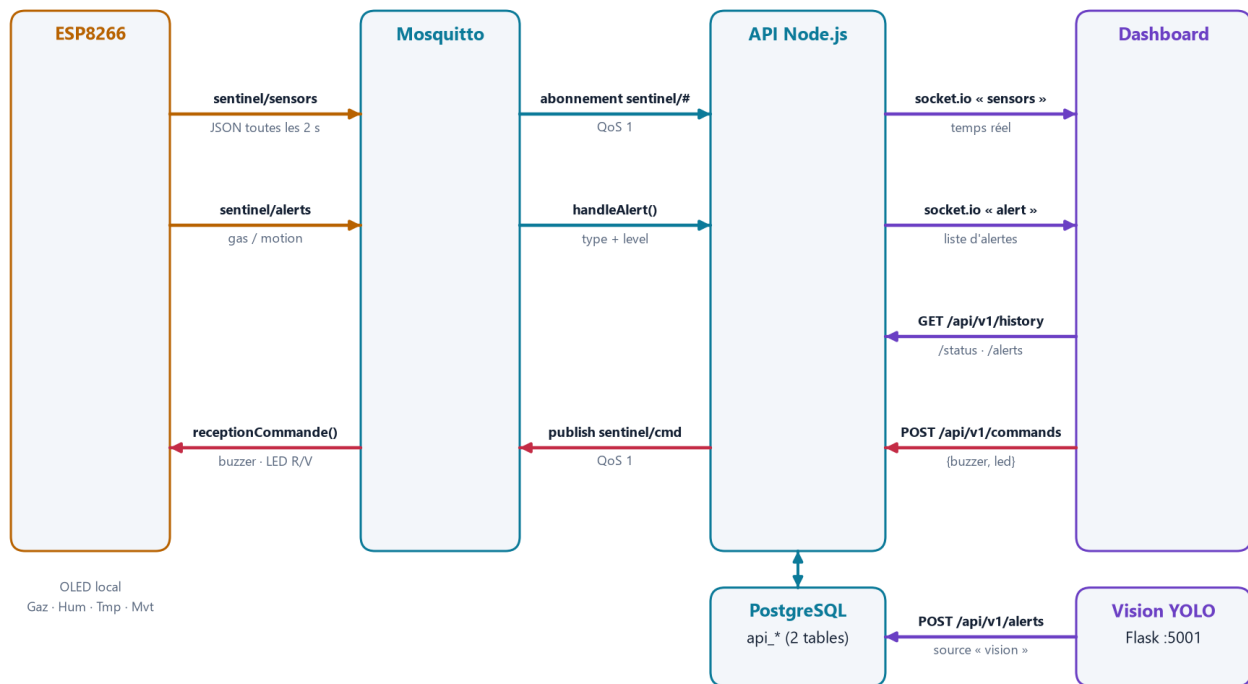


Figure 4 — Flux de données : topics MQTT, REST et événements socket.io.

6.1 Topics MQTT

Le backend s'abonne à <MQTT_BASE_TOPIC>/# et accepte deux contrats ; c'est la fin du topic qui décide du traitement (backend/src/index.js).

Topic	Sens	QoS	Charge utile	Traitement
sentinel/sensors	ESP → API	0	{temp, hum, gas, pir, ts}	INSERT api_measurements, socket « sensors »
sentinel/alerts	ESP → API	0	{type: gas motion temp, level, value}	INSERT api_alerts, socket « alert »
sentinel/cmd	API → ESP	1	{buzzer, led}	receptionCommande()
sentinelx/g11/telemetry	ESP → API	1	{seq, ts, t, h, gas, pir, rssi}	contrat sécurisé du kit
sentinelx/g11/event	ESP → API	1	{type, value, ts}	pir → motion critical / info
sentinelx/g11/status	ESP → API	1	online / offline (Last Will)	boîtier hors ligne immédiat

6.2 API REST (préfixe /api/v1)

Méthode	Route	Rôle	Validation / réponses
POST	/alerts	Alerte capteur ou IA, diffusée au dashboard	source ∈ {esp8266, vision}, type ≤ 50 car., value, timestamp ISO 8601, level optionnel ; 201 / 400
GET	/alerts?limit=20	Dernières alertes	limit entier 1-200
GET	/history?sensor=temp&limit=100	Historique des mesures	sensor ∈ {temp, hum, gas, pir}, limit 1-1000
GET	/status	État du boîtier et des actionneurs	online si message < 15 s
POST	/commands	Buzzer / LED	buzzer booléen, led ∈ {red, green, off} ; 503 si broker absent

```
POST /api/v1/alerts
{ "source": "vision", "type": "person_detected",
  "value": { "people": 1, "confidence": 0.87 },
  "timestamp": "2026-10-07T09:30:00Z", "level": "warn" }
```

Corps JSON limité à 10 Ko, erreurs 400 / 413 explicites, requêtes SQL paramétrées. Événements socket.io : sensors, alert, status (envoyé à chaque connexion, changement d'état en ligne / hors ligne et après chaque commande).

6.3 Modèle de données

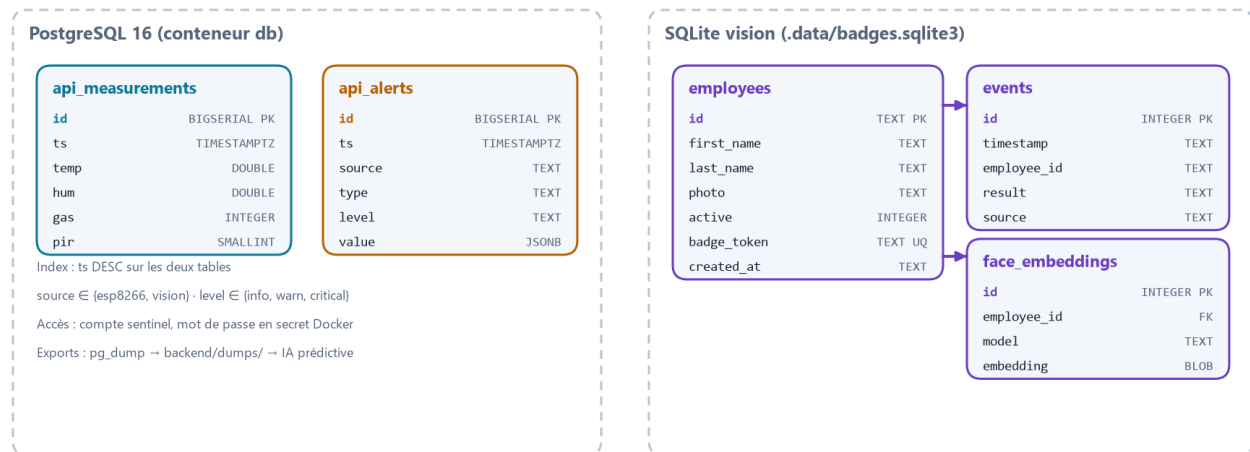


Figure 5 — Tables PostgreSQL du backend et base SQLite locale du service vision.

6.4 Dashboard

Application React 18 / Vite 5 (frontend/) : bandeau d'état global, tuiles capteurs colorées selon des seuils d'affichage, courbes Recharts des 30 derniers points, liste d'alertes, flux MJPEG de la webcam avec état de détection, panneau de commande buzzer / LED, chronologie de présence, et page « Employés & badges » (QR, vérification faciale par étapes avec consentement). La couche useSentinelData combine REST au démarrage et socket.io ensuite, avec un mode simulé (VITE_USE MOCK) pour développer sans matériel.

À COMPLÉTER PAR L'ÉQUIPE

Capture d'écran du dashboard en fonctionnement réel (courbes animées, alerte, flux webcam).

7. Matrice de sécurité

Elle reprend et complète security/docs/matrice-securite.md. Chaque ligne relie une menace à une contre-mesure, à son responsable et à la preuve présentée au jury.

7.1 Chiffrement et gestion des certificats

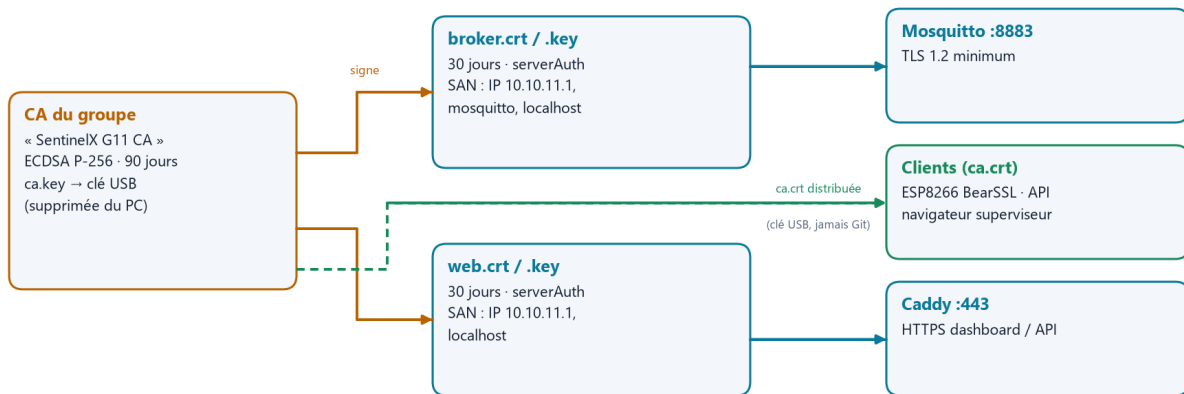


Figure 6 — PKI du groupe : une CA ECDSA, deux certificats serveur à IP dans le SAN.

- **ECDSA P-256 plutôt que RSA 2048** : poignée de main d'environ 1 s sur l'ESP8266 contre 3 à 5 s, et beaucoup moins de RAM.
- **IP dans le SubjectAltName** : BearSSL compare l'adresse demandée au certificat ; un faux broker sans certificat signé par la CA du groupe est refusé.
- **Durées courtes** : CA 90 jours, certificats 30 jours ; régénération par `make-pki.sh --force`.
- **Clé de la CA hors du PC** : stockée sur la clé USB du responsable cyber après génération.
- **Mosquitto** : `tls_version tlsv1.2, allow_anonymous false`, fichier `passwd` haché, ACL par compte, `max_connections 10, message_size_limit 1024`.

7.2 Durcissement du serveur

Domaine	Mesure	Fichier	Preuve attendue	État
Pare-feu	UFW deny incoming / allow outgoing / deny routed ; 443, 8883, 67, 53, 123 sur wlan0 ; 22 en limit	<code>hardening/ufw-rules.sh</code>	ufw status verbose ; liste des ports en écoute	Fourni
SSH	PasswordAuthentication no, PermitRootLogin no, AllowUsers, MaxAuthTries 3, pas de forwarding ; fail2ban	<code>hardening/sshd-sentinel.conf</code>	sshd -T ; tentative par mot de passe refusée	Fourni
Docker daemon	no-new-privileges, icc false, live-restore, logs json-file 3 × 10 Mo	<code>hardening/docker-daemon.json</code>	docker info	Fourni
Conteneurs	user non-root, cap_drop ALL, read_only, jamais privileged ni docker.sock	docker-compose (INFRA)	self-audit.sh section 4	Exigence
Ports Docker	publication sur 10.10.11.1 seulement, base jamais publiée	docker-compose (INFRA)	ss -lntu	Exigence
Secrets	secrets Docker (*_FILE), .env et certificats ignorés par Git	<code>backend/src/config.js</code>	self-audit.sh section 6	Fourni
Audit OS	lynis audit system avant / après	—	écart de score	[À COMPLÉTER]

7.3 Contrôle d'accès applicatif

Compte	Peut écrire	Peut lire
esp-g11 (boîtier)	sentinel(x)/.../telemetry, event, status	.../cmd uniquement
api (backend)	.../cmd	tout le groupe (#)
superviseur (Caddy)	POST /api/v1/commands via HTTPS	dashboard, API, flux vidéo
clé superviseur (vision)	routes employés / badges	fiches et journal d'accès

7.4 Analyse STRIDE

Menace	Scénario sur SENTINEL-X	Contre-mesure	Résiduel
S — Usurpation	Faux broker sur le Wi-Fi (evil twin), faux boîtier publiant des mesures	Trust anchor CA du groupe ; comptes MQTT ; WPA2	Faible
T — Altération	Commande forgée sur cmd, mesure modifiée en transit, injection SQL	ACL, TLS, validation des corps, SQL paramétré	Faible
R — Répudiation	Qui a déclenché le buzzer ?	Journal Caddy (identifiant) ; à ajouter : tracer chaque commande dans api_alerts	Moyen
I — Divulgarion	Sniffing des mesures, flux caméra ouvert, secrets dans Git, CORS *	MQTTS, vision sur 127.0.0.1, .gitignore, origines explicites	Moyen (historique F1)
D — Déni de service	Flood MQTT ou HTTP, saturation des 10 connexions, désauthentification Wi-Fi	Limites Mosquitto, express.json 10 Ko, UFW limit ; à ajouter : rate limit	Moyen
E — Élévation	Évasion de conteneur, brute-force SSH	non-root, cap_drop, no-new-privileges ; clés SSH, fail2ban	Faible

7.5 Failles relevées dans le code (audits du 6 et du 8 octobre)

#	Faille	Gravité	Statut
1	Service vision sur 0.0.0.0:5001 en HTTP sans authentification	Élevée	Corrigé : 127.0.0.1
2	Access-Control-Allow-Origin: * sur la vision	Moyenne	Corrigé : FRONTEND_ORIGINS
3	Serveur de développement Flask pour la démo	Faible	Atténué : local uniquement
4	Dashboard servi par Vite (host: true, HTTP)	Moyenne	Exigence : build statique derrière Caddy
5	POST /api/v1/commands sans authentification	Moyenne	Exigence : API via proxy avec identifiant
6	Clé Wi-Fi en clair dans le firmware (F1)	Élevée	[À COMPLÉTER] : clé changée ?
7	CORS_ORIGIN=* par défaut côté backend	Moyenne	À régler : URL du dashboard
8	Pas de limitation de débit sur l'API	Moyenne	À ajouter : rate limit Caddy ou express-rate-limit

7.6 Preuve matérielle du chiffrement

Le jury attend une preuve matérielle que les flux de la table sont chiffrés. Elle se construit sur **notre propre trafic**, au niveau conceptuel suivant :

- **Observation du trafic de la table** : sur le port 8883, seuls des enregistrements TLS chiffrés sont visibles ; aucune mesure JSON n'apparaît en clair.
- **Contre-preuve** : l'ancien mode MQTT en clair (1883) est désactivé côté broker ; une connexion non chiffrée ou anonyme est refusée.
- **Validation du certificat** : le boîtier et l'API n'acceptent que le certificat signé par la CA du groupe (IP dans le SAN, dates de validité en cours).
- **Surface d'exposition** : la liste des ports en écoute sur l'interface de table se limite à 443 et 8883.

À COMPLÉTER PAR L'ÉQUIPE

Joindre les preuves horodatées : capture d'écran du trafic 8883 chiffré, refus d'une connexion anonyme ou non chiffrée, liste des ports en écoute, sortie des scripts de contrôle du kit CYBER.

8. IA vision : détection de présence

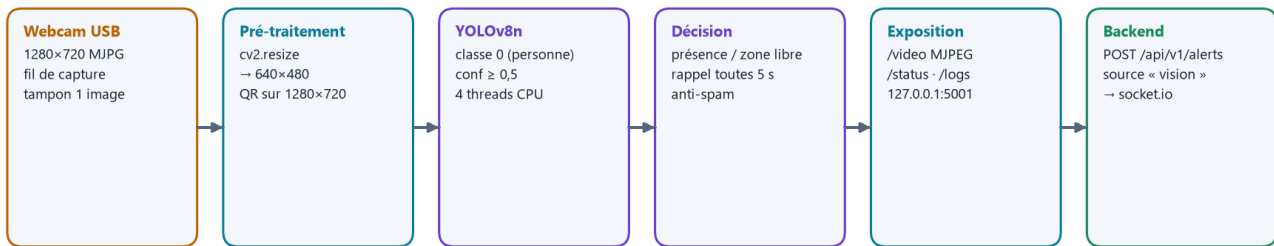


Figure 7 — Chaîne de traitement de webcam-detection/stream_detection.py.

8.1 Modèle et réglages

Paramètre	Valeur	Justification
Modèle	YOLOv8n (Ultralytics, yolov8n.pt, 3,2 M paramètres)	Plus petit modèle YOLOv8, temps réel sur CPU de portable
Classes	0 = person uniquement (classes=[0])	Seule la présence humaine est recherchée ; moins de faux positifs
Seuil de confiance	0,5 (conf=0.5) ; menace HIGH au-delà de 0,8 côté dashboard	Confiances observées 0,60 à 0,93 sur personne réelle
Entrée	capture 1280×720 MJPG, redimensionnée en 640×480	QR lisibles en haute définition, YOLO sur image réduite
Parallélisme	fil de capture (tampon d'une image), fil d'analyse, fil de rendu 20 img/s	La vidéo ne attend pas l'inférence
CPU	YOLO_THREADS=4, OMP_WAIT_POLICY=PASSIVE	Laisse du CPU à Docker et au navigateur
Journal	présence détectée / zone libre, rappel toutes les 5 s	Anti-spam ; « présence » et non « intrus » sans badge

8.2 Performances

Mesure	Valeur	Source
Inférence ponctuelle sur le serveur réel	25 ms	webcam-detection/COMPTE_REN DU_WEBCAM.md
Moyenne glissante sur 30 images (MacBook Air)	≈ 30 ms	webcam-detection/CHANGELOG.md
Âge de l'image affichée après optimisation (Core Ultra 7 155U)	10 à 30 ms (contre 150 à 270 ms)	CHANGELOG.md
Cadence du flux HTTP	≈ 18 images/s	CHANGELOG.md
Banc du dossier : 60 trames 640×480, 4 threads, PC sur batterie	216,4 ms d'inférence moyenne · p95 total 337,3 ms	livrables/src/analyse_ia.py --yolo
Mesure finale sur 1 minute, secteur, profil performances	[À COMPLÉTER] min / moy / max	équipe IA

Le banc réalisé pour ce dossier, sur batterie (mode économie d'énergie actif), dépasse nettement les 100 ms par trame. L'objectif est atteint dans les mesures de l'équipe (25 à 30 ms) mais dépend fortement de l'alimentation du portable. Pour la démonstration : portable sur secteur, profil « performances », et en cas de dépassement, trois leviers par ordre d'effort : réduire `imgsz` à 480 ou 320, n'analyser qu'une image sur deux, exporter le modèle en OpenVINO (`model.export(format="openvino")`) pour exploiter l'iGPU / NPU Intel du Core Ultra.

8.3 Intégration et vie privée

- Sorties : `/video` (MJPEG annoté), `/status` (detection, personnes, latence_ms, fps, analyse_ms, age_image_ms, confiance), `/logs` (50 événements).

- **À brancher** : envoi automatique de POST `/api/v1/alerts` (source: "vision", type: "person_detected") à chaque début de présence, pour que la détection apparaisse dans l'historique et déclenche l'alerte du dashboard.
- Badges QR et reconnaissance faciale (InsightFace buffalo_s) : enregistrement sur consentement, empreintes stockées localement dans SQLite, routes protégées par clé superviseur. Données biométriques au sens du RGPD : à limiter à la démonstration et à effacer ensuite.

9. IA prédictive : maintenance par détection d'anomalies

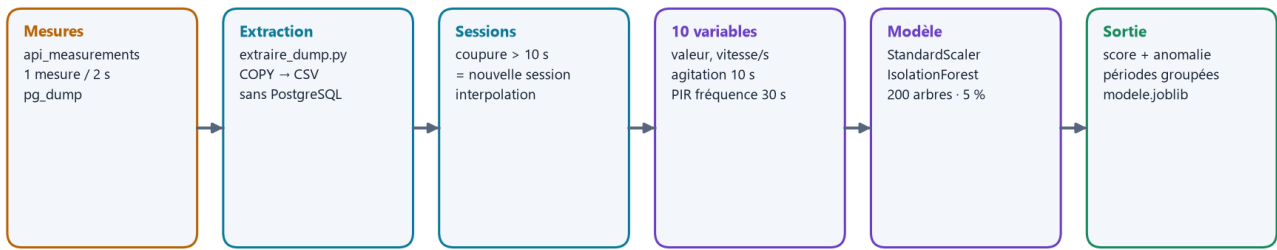


Figure 8 — Chaîne de traitement (branche IA-maintenance-predictive).

9.1 Pourquoi pas de seuils statiques

Un seuil du type `if temp > 40` ne voit qu'une valeur instantanée : il se déclenche trop tard (la surchauffe est déjà là) et ne voit pas une combinaison de signaux individuellement normaux. Le sujet cite l'exemple d'une hausse lente de température corrélée à une micro-dérive du gaz. Le modèle apprend donc le fonctionnement normal du boîtier sur ses propres données et repère les mesures qui s'en écartent, en regardant la **dynamique** des capteurs plutôt que leur seul niveau.

9.2 Variables (features)

Variable	Calcul	Ce qu'elle capte
temp, hum, gas	valeur brute interpolée	niveau
*_vitesse	différence / Δt (unités par seconde), par session	montée ou chute brutale
*_agitation	écart-type glissant sur 5 mesures (10 s)	instabilité, capteur perturbé
pir_frequence	moyenne glissante du PIR sur 15 mesures (30 s)	présence prolongée inhabituelle

Une **session** commence après plus de 10 s sans mesure (boîtier coupé) : aucune vitesse n'est calculée à travers une coupure. Les variables sont standardisées (`StandardScaler`) puis passées à un `IsolationForest(n_estimators=200, contamination=0.05, random_state=42)`.

9.3 Entraînement sur les données réelles

Indicateur	Valeur
Jeu de données	sentinel-2026-10-08_1225.sql : 5 429 mesures, 6 sessions, du 07/10/2026 14:10 au 08/10/2026 12:11
Plages observées	temp 0,0–38,2 °C · hum 0,0–100,0 % · gaz 0–1 024 (ADC)
Mesures jugées anormales	272 (5,0 %), regroupées en 24 périodes
Temps d'entraînement	1,11 s (CPU portable)
Temps de prédiction	24,9 μs par mesure, compatible avec un scoring en ligne toutes les 2 s

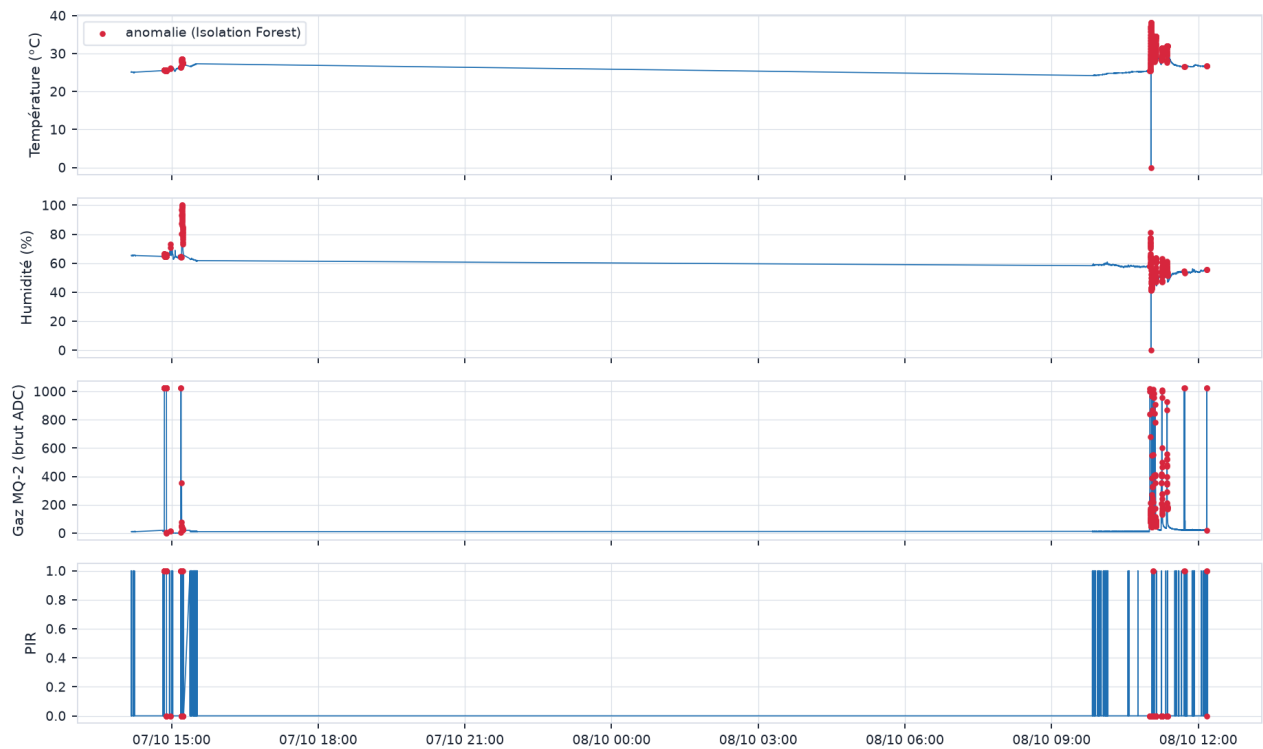


Figure 9 — Mesures réelles et points jugés anormaux par l'Isolation Forest (rouge).

Période (heure de Paris)	Mesures	Temp. max	Hum. max	Gaz max
07/10 14:51:21 → 14:53:55	78	25,7 °C	65,9 %	1024
08/10 11:00:45 → 11:02:41	54	38,2 °C	81,0 %	1018
07/10 15:12:57 → 15:13:43	24	28,2 °C	99,4 %	30
08/10 11:07:37 → 11:08:11	18	34,6 °C	63,7 %	94
08/10 11:15:01 → 11:15:35	17	31,5 °C	63,3 %	1010

Les plus longues périodes correspondent aux essais menés sur le boîtier (gaz de briquet, souffle chaud, humidité) : le modèle les isole sans qu'aucun seuil n'ait été écrit.

À COMPLÉTER PAR L'ÉQUIPE

Annoter ces périodes avec le journal des essais réels (heure, action, opérateur) pour confirmer qu'il s'agit bien de tests et estimer le taux de faux positifs en fonctionnement calme.

9.4 Évaluation par injection de dérives lentes

Faute d'incidents réels étiquetés, l'évaluation injecte 6 dérives synthétiques de 90 s dans les données réelles (même graine aléatoire pour toutes les méthodes) et compte les dérives détectées. Facteur $\times 1$: +0,05 °C et +2 unités de gaz par mesure.

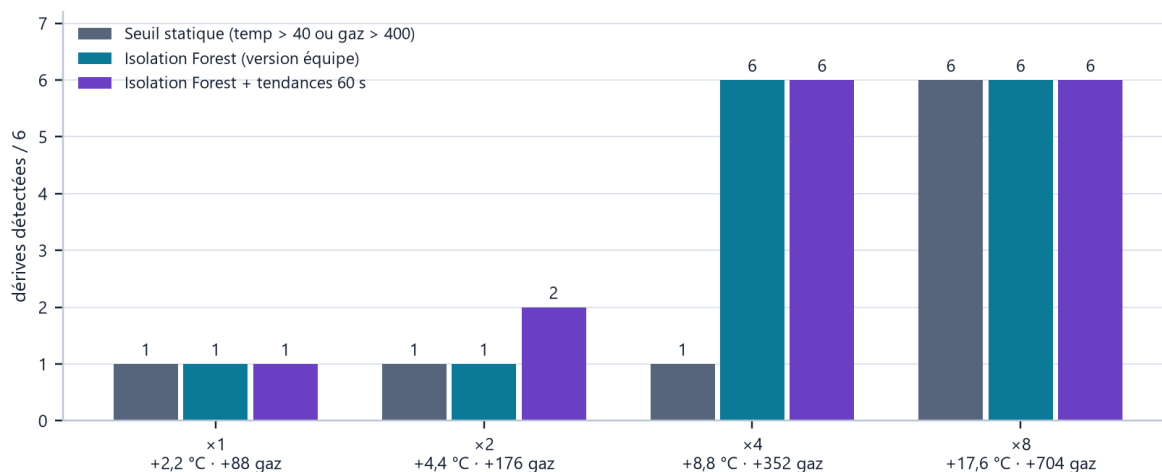


Figure 10 — Dérives détectées sur 6 selon leur amplitude.

Amplitude sur 90 s	Seuil statique	IF équipe	IF + tendances 60 s	Rappel par point (équipe / +tendances)
×1 : +2,2 °C, +88 gaz	1/6	1/6	1/6	2 % / 2 %
×2 : +4,4 °C, +176 gaz	1/6	1/6	2/6	2 % / 4 %
×4 : +8,8 °C, +352 gaz	1/6	6/6	6/6	38 % / 79 %
×8 : +17,6 °C, +704 gaz	6/6	6/6	6/6	88 % / 94 %

- Dès une dérive de +8,8 °C et +352 unités de gaz en 90 s, l'Isolation Forest détecte les 6 épisodes, le seuil statique un seul (celui qui partait déjà d'une valeur élevée) : le modèle réagit à la **vitesse**, le seuil attend le **niveau**.
- Les dérives très lentes (×1, ×2) restent dans le bruit réel des capteurs : les données d'entraînement contiennent les essais volontaires (gaz jusqu'à 1024, humidité 100 %), que le modèle a appris comme « possibles ».
- Ajouter des tendances sur 60 s double le rappel par point à ×4 (38 % → 79 %) sans augmenter le taux d'alertes sur les données réelles (5 %).

9.5 Limites et suite

- Entraîner sur une session de référence calme, sans essais, et garder les sessions d'essais pour la validation (détection de nouveauté plutôt que d'anomalies).
- Ajouter les variables de tendance 60 s (évaluées ici) et la corrélation glissante température / gaz.
- Scorer en ligne : charger `modele_anomalies.joblib` dans un petit service Python abonné à `sentinel/sensors` et publier les anomalies par POST `/api/v1/alerts` (nouvelle source « predictive » à autoriser dans `backend/src/routes.js`).
- Réentraîner après chaque changement physique du boîtier (emplacement, MQ-2 remplacé).

10. Gabarit de rapport d'audit de sécurité

Cadre

Ce chapitre est un **gabarit vierge** que l'équipe complète après la journée d'audit du jeudi. Il recense les points de contrôle défensifs de notre propre table et les résultats observés. Il ne décrit aucune procédure offensive : l'audit se déroule dans le cadre fixé par les coachs.

10.1 Identification de l'audit

Champ	Valeur
Date et créneau	[À COMPLÉTER]
Périmètre audité	Table du groupe : boîtier ESP8266, PC serveur, Wi-Fi de table, pile Docker
Auditeurs / auditrices	[À COMPLÉTER]
Version du système audité	[À COMPLÉTER] (commit Git, version firmware)
Référent CYBER du groupe	[À COMPLÉTER]

10.2 Grille des points de contrôle

Pour chaque point : **C** conforme, **NC** non conforme, **NA** non applicable. La colonne « Preuve » renvoie à une pièce jointe (capture, sortie de commande de contrôle, photo).

#	Domaine	Point de contrôle (état attendu)	Résultat	Preuve	Gravité
C1	Chiffrement en transit	Flux ESP8266 ↔ broker en MQTTS (TLS 1.2+), aucun flux capteur en clair			
C2	Chiffrement en transit	Dashboard et API servis en HTTPS uniquement			
C3	Certificats	Certificats signés par la CA du groupe, IP dans le SAN, en cours de validité			
C4	Authentification broker	Connexions anonymes refusées, un compte par client, ACL par topic			
C5	Surface d'exposition	Seuls 443 et 8883 accessibles depuis le Wi-Fi de table			
C6	Pare-feu hôte	Politique « refus par défaut » en entrée et en routage			
C7	Administration	SSH par clé uniquement (ou désactivé), pas de connexion root			
C8	Conteneurs	Utilisateur non-root, capacités retirées, aucun conteneur privilégié			
C9	Secrets	Aucun secret dans Git ; secrets Docker / fichiers ignorés			
C10	Réseau de table	WPA2, isolation des clients, pas de routage vers le campus			
C11	Applicatif	Validation des entrées API, requêtes SQL paramétrées, CORS restreint			
C12	Supervision	Journaux conservés et bornés, alertes de disponibilité actives			
C13	Sauvegarde	Export de la base réalisé et restauration testée			
C14	Physique	Boîtier fermé, port USB non exposé, câbles protégés			

10.3 Synthèse des constats

#	Constat	Gravité	Recommandation défensive	Responsable	Échéance	Statut
N1						
N2						
N3						
N4						
N5						
N6						

10.4 Échelle de gravité

Niveau	Définition
Critique	Compromission possible de la confidentialité ou de l'intégrité des mesures ou des commandes ; correction avant la démonstration.
Élevée	Écart à une exigence du sujet (chiffrement, durcissement) ; correction prioritaire.
Moyenne	Affaiblit la défense en profondeur sans exposition directe ; correction planifiée.
Faible	Bonne pratique non appliquée, impact limité.
Information	Observation sans impact de sécurité, à documenter.

10.5 Indicateurs avant / après durcissement

Indicateur	Avant	Après	Commentaire
Ports accessibles depuis le Wi-Fi de table			
Part des flux chiffrés (MQTT, HTTP)			
Comptes broker anonymes autorisés			
Conteneurs exécutés en root			
Secrets présents dans le dépôt Git			
Score de l'outil d'audit de configuration de l'OS			

10.6 Recommandations défensives génériques

- **Chiffrer tous les flux** entre le boîtier, le broker, l'API et le navigateur, et refuser toute connexion non chiffrée.
- **Authentifier chaque client** (boîtier, API, superviseur) avec un compte distinct et des droits limités au strict nécessaire.
- **Réduire la surface d'exposition** : refus par défaut au pare-feu, publication des ports Docker sur l'interface de table seulement, services internes liés à 127.0.0.1.
- **Appliquer le moindre privilège** aux conteneurs et aux services : non-root, capacités retirées, systèmes de fichiers en lecture seule quand c'est possible.
- **Protéger les secrets** : hors du dépôt Git, rotation après toute fuite, clé de la CA conservée hors ligne.
- **Administrer par clés** asymétriques, sans mot de passe ni compte root à distance.
- **Superviser et journaliser** les événements de sécurité (connexions refusées, redémarrages), avec une rétention bornée.
- **Tenir à jour** le système hôte, les images Docker et les bibliothèques du firmware.

À COMPLÉTER PAR L'ÉQUIPE

Remplir les tableaux 10.1 à 10.5 après la journée d'audit, joindre les preuves en annexe et reporter les corrections effectuées (statut « corrigé », commit associé).

11. Supervision et maintien en condition opérationnelle

Le PC serveur reçoit une mesure toutes les 2 s, des images webcam en continu et les connexions du dashboard. Le MCO vise à garder la machine hôte disponible pendant toute la démonstration et à détecter tôt une dérive (saturation CPU, disque rempli par les journaux, boîtier hors ligne).

11.1 Indicateurs suivis

Indicateur	Source	Seuil d'attention	Action
CPU hôte	Gestionnaire des tâches / docker stats	> 85 % pendant 1 min	Réduire la résolution d'analyse YOLO, fermer les applications inutiles
RAM hôte	docker stats	> 80 %	Redémarrer le service le plus consommateur
Espace disque / volumes Docker	docker system df	< 5 Go libres	Purger les images inutilisées, exporter puis réduire l'historique
Volume des journaux Mosquitto	pilote json-file	3 × 10 Mo (rotation)	Rotation automatique, niveau de log réduit
Débit MQTT	compteur des mesures reçues par l'API	< 1 mesure / 5 s	Vérifier le Wi-Fi et l'alimentation du boîtier
Boîtier en ligne	GET /api/v1/status	aucun message depuis 15 s	Badge « hors ligne » au dashboard ; vérifier le boîtier
Latence vision	/status du service vision	> 100 ms par trame	Secteur, profil performances, imgsz réduit
Validité des certificats	dates du certificat	< 7 jours restants	Régénérer la PKI avant expiration
Redémarrages de conteneurs	docker compose ps	> 2 par heure	Lire les journaux du service concerné

11.2 Résilience déjà en place

- **Redémarrage automatique** : restart: unless-stopped sur les trois services (backend/docker-compose.yml).
- **Ordre de démarrage** : l'API attend que PostgreSQL soit sain (healthcheck pg_isready) et réessaie la connexion à la base 20 fois.
- **Reconnexion MQTT** toutes les 2 s côté API ; un message mal formé est ignoré sans interrompre la réception.
- **Mode dégradé du boîtier** : sans serveur, l'OLED, le buzzer et les LED continuent de signaler localement gaz et présence.
- **Persistance** : volumes Docker nommés pour la base et le broker ; exports réguliers dans backend/dumps/.

11.3 Gestion des journaux et des données

Donnée	Politique proposée
Journaux des conteneurs	json-file, 3 fichiers de 10 Mo maximum par conteneur
Journal Mosquitto	sortie standard (gérée par Docker), connexions et refus conservés
Mesures api_measurements	conservation 30 jours en démonstration, export avant purge
Alertes api_alerts	conservation intégrale pendant le workshop (traçabilité)
Données biométriques (vision)	consentement explicite, effacement à la fin du workshop

11.4 Procédures d'exploitation

Symptôme	Vérification	Remédiation
Dashboard « hors ligne »	Statut des conteneurs, journaux de l'API	Redémarrer l'API ; vérifier la connexion au broker
Plus de mesures	Boîtier alimenté, OLED « MQTT OK », Wi-Fi de table actif	Redémarrer le boîtier ; vérifier l'heure (NTP) pour le TLS
Vision lente	Latence affichée par /status	Brancher sur secteur, réduire la résolution d'analyse
Disque presque plein	Occupation des volumes Docker	Exporter la base, purger les anciennes mesures
Certificat expiré	Dates du certificat broker / web	Régénérer la PKI, redéployer ca.crt sur le boîtier

À COMPLÉTER PAR L'ÉQUIPE

Relevés réels pendant une répétition de démonstration de 10 minutes : CPU et RAM moyens et maximaux, taille des journaux, nombre de mesures reçues, latence vision.

12. Conclusion et perspectives

SENTINEL-X relie de bout en bout un boîtier de table ESP8266, une pile conteneurisée sur le PC portable serveur, une vision par ordinateur locale et un modèle de détection d'anomalies entraîné sur les mesures réelles du boîtier. L'option B tire parti de la puissance du portable pour l'IA et garde le boîtier compact.

La sécurité repose sur des principes défensifs simples et vérifiables : chiffrement de tous les flux, authentification de chaque client, surface d'exposition réduite à deux ports, moindre privilège pour les conteneurs et secrets hors du dépôt.

Perspectives

- Scorer les anomalies en ligne et les publier comme alertes « predictive » sur le dashboard.
- Accélérer la vision (export OpenVINO) pour tenir l'objectif de 100 ms sur batterie.
- Signer les mises à jour du firmware et tracer chaque commande d'actionneur.
- Passer à plusieurs boîtiers par table avec un compte MQTT et un certificat par boîtier.

« Sentinel-X : la sécurité à la bordure. »

Annexe — Glossaire

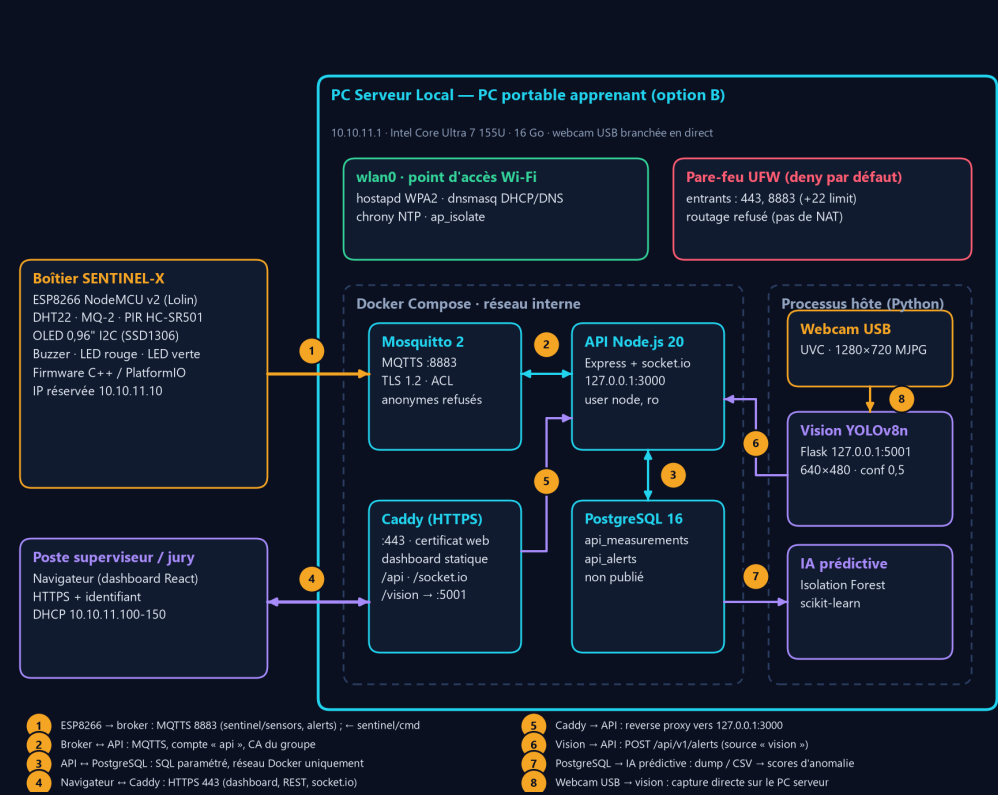
Terme	Définition
ACL	Liste de contrôle d'accès : droits de lecture / écriture d'un compte sur les topics MQTT.
CA	Autorité de certification : signe les certificats du broker et du serveur web du groupe.
Isolation Forest	Modèle non supervisé qui isole les observations rares par partitions aléatoires.
MCO	Maintien en condition opérationnelle de la plateforme.
MQTT / MQTTS	Protocole de messagerie publication / abonnement ; MQTTS = MQTT sur TLS (port 8883).
SAN	Subject Alternative Name : noms et adresses IP pour lesquels un certificat est valide.
STRIDE	Méthode de modélisation des menaces : usurpation, altération, répudiation, divulgation, déni de service, élévation de privilèges.
TLS	Protocole de chiffrement et d'authentification des flux réseau.
YOLOv8n	Version « nano » du détecteur d'objets YOLO d'Ultralytics.

SENTINEL-X

Boîtier IoT durci · IA locale temps réel · pile conteneurisée chiffrée de bout en bout

- Option B · PC portable serveur
- Groupe GX
- ESP8266 → MQTTS → Docker

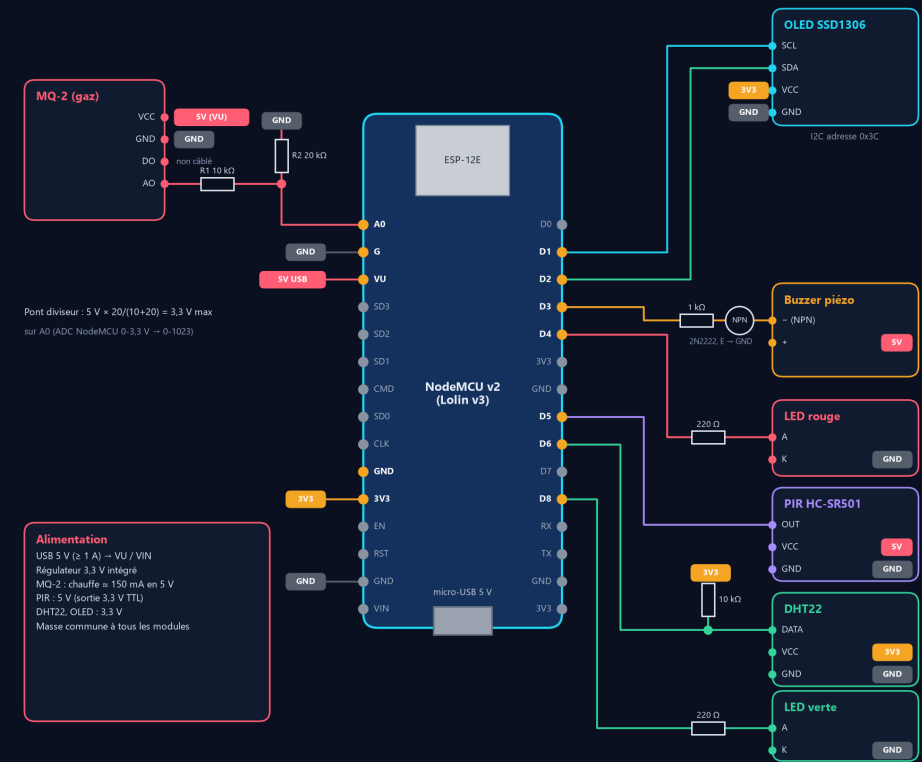
ARCHITECTURE EDGE-TO-SERVER



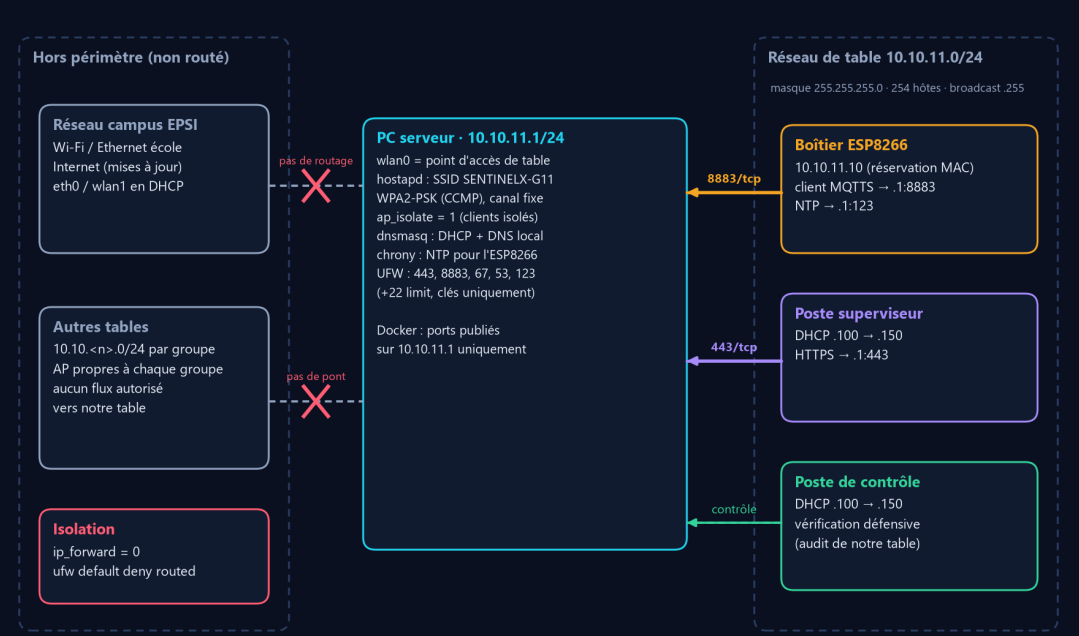
CHIFFRES CLÉS

- 2
ports exposés sur le Wi-Fi de table : 443 (HTTPS) et 8883 (MQTTs)
- TLS 1.2
ECDSA P-256 · CA du groupe · comptes MQTT et ACL par topic
- 5 429
mesures réelles du boîtier analysées (07/10 → 08/10)
- 6/6 vs 1/6
dérives lentes détectées : Isolation Forest vs seuil statique

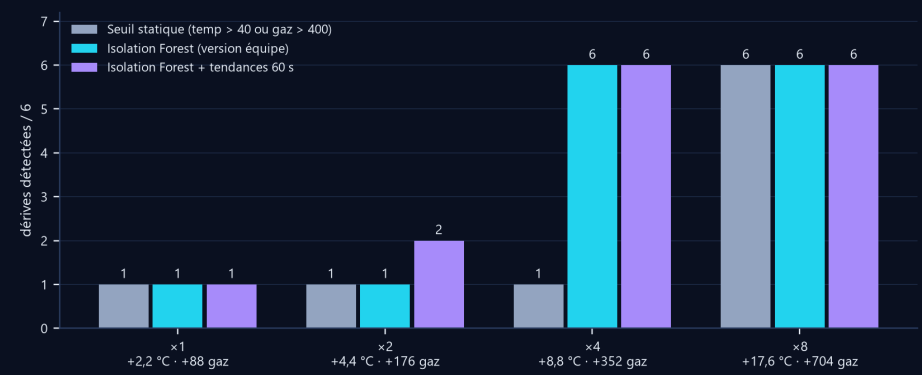
LE BOÎTIER ESP8266



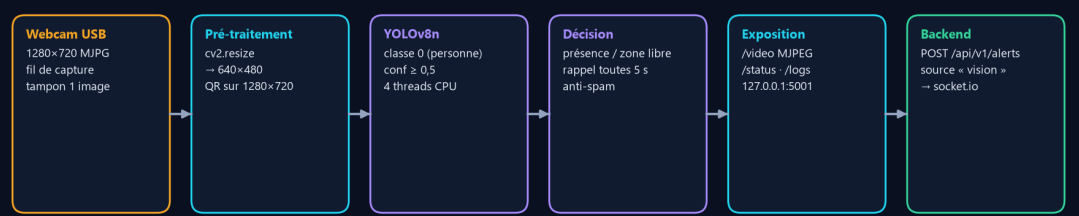
RÉSEAU DE TABLE ÉTANCHE



IA PRÉDICTIVE : ISOLATION FOREST



VISION : YOLOV8N SUR LA WEBCAM USB



SÉCURITÉ DÉFENSIVE

- MQTTS uniquement, anonymes refusés · UFW refus par défaut
- SSH par clé · conteneurs non-root, cap_drop ALL
- Secrets hors Git, clé de la CA hors ligne

Sentinel-X : la sécurité à la bordure.